

# Chapter 8

## The Career Tree: How Skills Unlock Roles

Use the "keys and doors" idea to decide what to learn next · name the seven tiers of the career tree and the roles each one unlocks · read a skills matrix to see what each role really needs

Approved chapter · v1

# Contents

## Chapter 8. The Career Tree: How Skills Unlock Roles

Why this matters

In plain English

8.1 Skills are keys; roles are doors

8.2 The tiers, and what each unlocks

8.3 The skills matrix: what each role really needs

8.4 A day in the life of each role

8.5 Decoding a job description

8.6 What the roles pay, and how to read salary figures

8.7 Reader pathways: your route through this book

8.8 Entry routes: fresher, career switcher, internal move

8.9 Using the tree: pick your door, then learn backward

Common mistakes and how to spot them

In the real world: Farah's internal move

Tools

The project: your door plan

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

# Chapter 8. The Career Tree: How Skills Unlock Roles

Part I — The Map

## Chapter at a glance

**You will learn to:** use the "keys and doors" idea to decide what to learn next · name the seven tiers of the career tree and the roles each one unlocks · read a skills matrix to see what each role really needs · picture a normal working day in each of the ten roles · decode a job description line by line and score your own fit · read salary figures carefully, and check them yourself · choose a reading pathway through this book for your goal · plan an entry route as a fresher, a career switcher, or through an internal move.

**Before you start:** Chapter 7 (the four questions, the ten roles, and how teams are organized).

**Time needed:** 3–4 hours, including the exercises. Allow another 2–3 hours for the project, which uses real job postings.

**Tools:** a spreadsheet (Excel or Google Sheets) or paper, and a job portal for the project. Nothing to install.

**Practice data:** a fictional job description from Riverstone Supplies, the skills matrix in section 8.3, and salary figures published by PayScale and Indeed for India, as retrieved in September 2026. Every calculation shown is checked.

---

## Why this matters

"What should I learn next?" is the most common question in every data community, and it's usually asked backwards. People ask about tools in isolation: *Should I learn Spark? Is Tableau worth it? Do I need deep learning?* The useful question is about **doors**: *which job would this skill let me apply for that I can't apply for today?*

Asked backwards, the question leads to months of learning things no employer will ask you about yet, while the three skills that would get you shortlisted stay half-learned. Asked the right way round, it turns an endless list of topics into a short, ordered plan with a job at the end of each step.

This chapter gives you that plan. It arranges the ten roles from Chapter 7 into a tree of tiers, shows exactly which skills each role needs, lets you see what each job looks like on an ordinary Tuesday, and teaches you to read a job description the way a hiring manager wrote it. By the end, you'll be able to name your next door, and the chapters of this book that open it.

---

## In plain English

---

Picture an office building where every room is a job. Each room has a locked door, and each lock needs a particular set of keys.

- Each **key** is a skill: SQL, a spreadsheet, a BI tool, Python, statistics, the ability to write a clear requirement.
- Each **door** is a role you could be hired into. When you hold the keys a door needs, you become a credible applicant: someone a hiring manager would take seriously.
- The building has **floors**. The ground floor's keys open the first rooms. Higher floors need the lower keys *and* some new ones. Nobody takes the lift straight to the top.
- On one floor, two corridors branch off in different directions, one toward science and one toward engineering. They're on the same floor: neither is "above" the other.
- You **keep your keys** as you climb. A data scientist still uses SQL every week.
- A **job description** is the notice on a door, listing the keys it needs. Some are required; some are "nice to have".

The building is the **career tree**. The floors are **tiers**. The rest of this chapter is its floor plan.

---

### 8.1 Skills are keys; roles are doors

---

The draft of this book introduced one idea that stays at the center of this chapter: **learn skills for the doors they open, not because they're popular**.

That idea changes how you study in three ways.

- **It gives every topic a reason.** You don't learn "data science" in the abstract. You learn the set of skills that makes you a credible applicant for *data scientist*, and each skill is a key you can see yourself adding.
- **It tells you when to stop.** When you hold the keys a door needs, you're ready to apply. You'll keep learning on the job, but you don't need to finish the whole book first.
- **It protects you from fashion.** A tool that's trending this year might open no door you want. A skill that appears in every job description for your target role is worth more than ten trending ones.

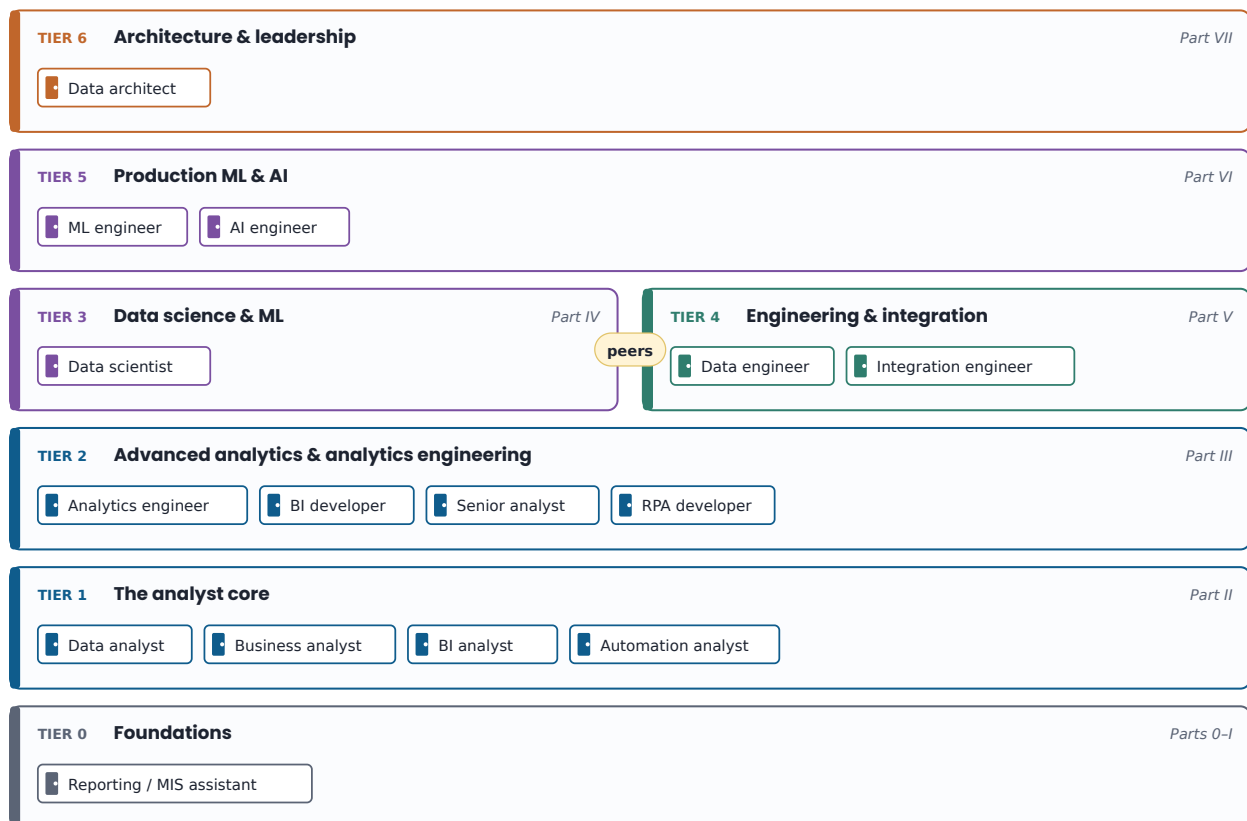
A **credible applicant** is someone who meets a role's must-have skills well enough to pass a screening and hold their own in a technical interview. It doesn't mean knowing everything, and it doesn't mean experience alone. Part VIII (from Chapter 68) shows what each interview round checks.

---

### 8.2 The tiers, and what each unlocks

---

The career tree has seven tiers, numbered from 0. They match the parts of this book, in order, so the book is the tree.



Read from the bottom up. Each role sits at the first tier that makes you a credible applicant; you keep every skill below it.

Figure 8.1 – The career tree as tiers and doors. Tiers 3 and 4 sit side by side: they're peers, and you can climb either one first.

**Tier 0: Foundations (Parts 0 and I).** What data is, how computers store it, how a business runs on it, numbers without fear, thinking like an analyst, and the map you're reading now. *Unlocks:* reporting assistant and MIS (management information system) roles, where you prepare regular reports mostly by hand. Few people stay here long, and the skills matter mainly because everything above rests on them.

**Tier 1: The analyst core (Part II).** Excel and Google Sheets in depth, SQL, cleaning data, visualization, Power BI, Python for analysis, spreadsheet and report automation, statistics, business metrics, working with stakeholders, and the professional toolkit. *Unlocks:* **data analyst**, **business analyst**, **BI analyst** (an analyst who mainly builds reports and dashboards), **automation analyst**, and the domain versions: sales, finance, marketing, operations, supply chain, and HR analyst. This is the single biggest unlock in the field. If you master only one tier, make it this one.

**Tier 2: Advanced analytics and analytics engineering (Part III).** Advanced SQL and data modeling, Python written as maintainable software, experiments and causal inference, dbt, the computer science you need, and the command line. *Unlocks:* **senior analyst**, **analytics engineer**, **BI developer** (who designs the data models and semantic layers behind dashboards, not only the visuals), and **RPA developer**. Tier 2 is the branch point: the skills here feed both branches above it.

**Tier 3: Data science and machine learning (Part IV).** The math under the models, the machine learning workflow, supervised and unsupervised learning, honest evaluation, forecasting, text, recommendations, and a first look at deep learning. *Unlocks:* **data scientist**, and specialist roles such as forecasting analyst or applied scientist.

**Tier 4: Engineering and integration (Part V).** Ingestion, pipelines and orchestration, data quality, distributed compute, warehouses and lakehouses, streaming, data activation and APIs, and the cloud. *Unlocks:* **data engineer**, **integration engineer**, and platform roles.

**Tiers 3 and 4 are peers.** They sit at the same height on the tree. The book teaches science first because many analysts move toward it first, but plenty of excellent careers go from tier 2 straight into engineering. This is the fix for the first edition's tier order, which put engineering before science while the book taught them the other way round. Now tier numbers, part numbers, and the tree agree, and the figure shows the two branches side by side.

**Tier 5: Production ML and AI (Part VI).** Deep learning in depth, generative AI and large language models, building AI applications, MLOps, LLMOps, intelligent automation, and industry cases. *Unlocks:* **ML engineer**, **AI engineer**, and MLOps and ML platform roles. Most people arrive here from tier 3 (with engineering added) or from tier 4 (with machine learning added).

**Tier 6: Architecture and leadership (Part VII).** Designing whole systems, trade-offs, architecture patterns, automation architecture and governance, security and privacy, the economics of data platforms, data strategy, and leading as an architect. *Unlocks:* **data architect**, ML or AI architect, and the management ladder up to head of data or chief data officer.

## Where the automation roles sit

The automation and integration track from Chapter 7 doesn't fit neatly on one floor, because automation grows with every tier. Its roles appear at three levels:

| Role                 | Tier | Why there                                                                                              |
|----------------------|------|--------------------------------------------------------------------------------------------------------|
| Automation analyst   | 1    | Spreadsheet automation, scheduled reports, and low-code flows are analyst-core skills (Chapters 19–20) |
| RPA developer        | 2    | Needs maintainable, tested automation and command-line habits (Part III), plus RPA tools               |
| Integration engineer | 4    | Needs APIs, pipelines, data quality, and reverse ETL (Chapters 45–51)                                  |

## How the tree works: three rules

These rules from the first edition still hold, and the rest of the chapter depends on them.

1. **You keep every skill below your tier.** Climbing doesn't mean leaving skills behind. They accumulate.
2. **A role sits at the first tier that makes you credible for it.** You don't need tier 6 to apply for a tier 1 job. Learn up to the door you want, apply, and keep climbing on the job.
3. **Pick your door, then learn backward.** Choose the role you want next, find its tier, and learn the keys that door needs. Ignore everything higher for now. It will still be there when you arrive.

**Simplification note.** Real hiring is messier than a tree. A small company might hire a "data scientist" who mostly does tier 1 work, and a large bank might require tier 2 skills for an analyst post. The tiers describe where a role's *must-have* skills are taught, which is what you need for planning. Section 8.5 shows how to read what a particular employer really wants.

## 8.3 The skills matrix: what each role really needs

The tiers tell you roughly where a role sits. The skills matrix tells you exactly which keys each door needs. It lists 17 skills against the ten roles, and marks each skill as **core** (needed to be hired), **useful** (helps, often listed as a plus), or **not needed** to get in.

| Skill                             | DA | BA | BI | AE | DS | MLE | DE | AIE | AUT | ARC |
|-----------------------------------|----|----|----|----|----|-----|----|-----|-----|-----|
| Business and domain sense         | ●  | ●  | ○  | ○  | ●  | ○   | ○  | ○   | ●   | ●   |
| Communication and storytelling    | ●  | ●  | ○  | ○  | ●  | ○   | ○  | ○   | ○   | ●   |
| Spreadsheets (Excel, Sheets)      | ●  | ●  | ○  | ·  | ○  | ·   | ·  | ·   | ●   | ○   |
| SQL                               | ●  | ○  | ●  | ●  | ●  | ○   | ●  | ○   | ○   | ●   |
| BI and dashboards                 | ●  | ○  | ●  | ○  | ○  | ·   | ·  | ·   | ○   | ○   |
| Statistics                        | ○  | ·  | ·  | ·  | ●  | ○   | ·  | ○   | ·   | ·   |
| Python                            | ○  | ·  | ·  | ○  | ●  | ●   | ●  | ●   | ●   | ○   |
| Spreadsheet and report automation | ○  | ○  | ○  | ·  | ·  | ·   | ·  | ·   | ●   | ·   |
| Requirements and process mapping  | ○  | ●  | ○  | ○  | ·  | ·   | ·  | ○   | ●   | ●   |
| Data modeling (incl. dbt)         | ·  | ·  | ●  | ●  | ·  | ·   | ●  | ·   | ·   | ●   |
| Machine learning                  | ·  | ·  | ·  | ·  | ●  | ●   | ·  | ○   | ·   | ○   |
| Pipelines and orchestration       | ·  | ·  | ·  | ○  | ·  | ●   | ●  | ○   | ○   | ●   |
| Cloud and infrastructure          | ·  | ·  | ·  | ○  | ○  | ●   | ●  | ●   | ○   | ●   |
| APIs and integration              | ·  | ·  | ·  | ·  | ·  | ○   | ●  | ●   | ●   | ●   |
| LLMs and AI applications          | ·  | ·  | ·  | ·  | ○  | ○   | ·  | ●   | ○   | ○   |
| Git and version control           | ·  | ·  | ○  | ●  | ○  | ●   | ●  | ●   | ○   | ○   |
| System design and governance      | ·  | ·  | ·  | ○  | ·  | ○   | ○  | ○   | ○   | ●   |

Key: ● core · ○ useful · · not needed to get in. DA data analyst · BA business analyst · BI BI developer · AE analytics engineer · DS data scientist · MLE ML engineer · DE data engineer · AIE AI engineer · AUT automation analyst, RPA developer, or integration engineer · ARC data architect.

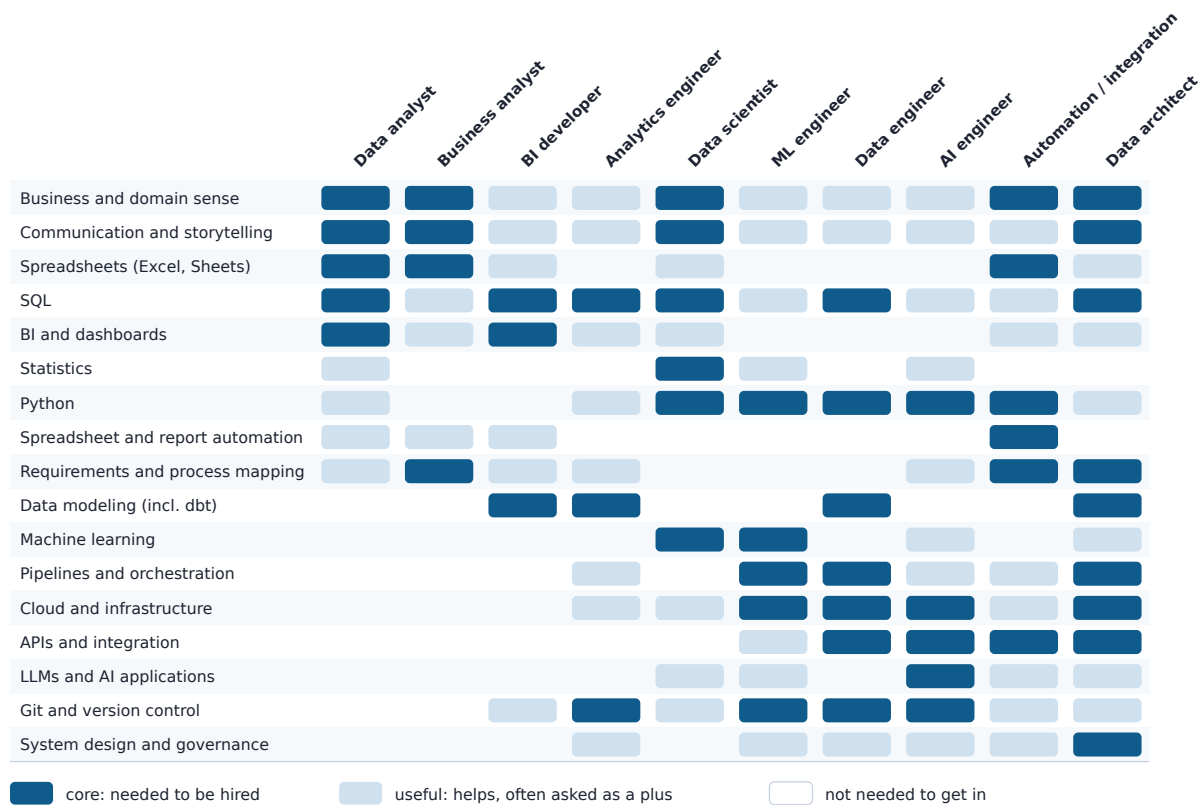


Figure 8.2 – The same matrix as a heatmap. Read down a column to see a role; read across a row to see which doors a skill helps open.

## Reading the matrix

**Read down a column** to see what a role needs. The data analyst column has five core skills: business sense, communication, spreadsheets, SQL, and BI and dashboards. That's the tier 1 core. The data engineer column has seven: SQL, Python, data modeling, pipelines, cloud, APIs and integration, and Git. The data architect column is the darkest of all, with nine core skills, because the architect's job is to understand how every part fits.

**Read across a row** to see which doors a skill helps open. Three rows are worth noticing.

- **SQL** is marked core or useful for all ten roles. This is the promise from Chapter 7 in matrix form.
- **Machine learning** is core for only two roles, data scientist and ML engineer. If neither is your target, it can wait.
- **Python** is core for five roles (data scientist, ML engineer, data engineer, AI engineer, and the automation and integration roles) and useful for three more. It's the most important skill to add after the analyst core.

**Compare two columns** to see how far apart two roles are. Data analyst and BI developer share seven skills that each role needs at least a little of; data analyst and data engineer share only four (business sense, communication, SQL, and Python). That's why analyst-to-BI-developer is a short move and analyst-to-data-engineer takes longer.

**Watch out: "useful" is not "optional forever."** A useful skill often decides between two otherwise equal candidates, and it's usually the next key you'll need on the job. Treat the useful cells in your target column as your second list, after the core ones.

**Try it.** Pick the role you're most curious about. Count its core skills, and tick the ones you could use at work today. The unticked ones are your next keys.

## 8.4 A day in the life of each role

A matrix tells you what a job needs. It doesn't tell you what the job *feels* like. Here's an ordinary working day in each role. The people and companies are fictional; the shape of each day is typical, though every company differs.

### Data analyst

Pooja works in the sales team of a consumer goods distributor. At 9:30 she checks that the morning's sales report refreshed and that yesterday's total matches the billing system. At 10:00 the regional manager asks why the West region missed its target; she spends two hours in SQL and a spreadsheet, finds that one large wholesaler delayed an order to next month, and writes a four-line answer. After lunch she builds a comparison of two discount schemes for Friday's review. At 4:30 a colleague reports a strange number on the dashboard, and she traces it to a customer entered twice. Most of her day is questions, SQL, checking, and explaining.

### Business analyst

Aditya works for a hospital chain that is replacing its appointment system. His morning is a workshop with front-desk staff, mapping how appointments are booked, changed, and cancelled today, including the paper register nobody mentioned in the first meeting. He turns the notes into a process map and eight user stories, each with acceptance criteria. After lunch he pulls last quarter's cancellation data with a short SQL query to show that most cancellations happen by phone, which changes the design. The day ends with a call with the software vendor to agree what "reschedule" means. Most of his day is conversation, documents, and small, sharp pieces of analysis.

### BI developer

Lakshmi works in a bank's reporting team. She starts by checking the overnight dataset refreshes in Power BI; one failed because a source table was renamed, so she fixes the query. Her main task is a new branch-performance dashboard: she designs the data model, writes measures, and makes sure the numbers match the finance team's official figures to the rupee. In the afternoon she reviews a colleague's report for slow visuals and confusing labels. Before leaving, she sets row-level security so branch managers see only their own branch. Her day is modeling, measures, performance, and access.

## Analytics engineer

Rohan works at an online retailer. His morning starts with a failing data test: an "orders" model suddenly has duplicate order IDs. He finds that a source system began sending corrections as new rows, fixes the model, and adds a test so it can't happen silently again. Then he meets the marketing and finance analysts to agree a single definition of "new customer". He writes the change in dbt, documents it, and opens a pull request that a colleague reviews. In the afternoon he helps an analyst turn a 300-line query into three reusable models. His day is SQL, tests, Git, and agreeing definitions.

## Data scientist

Meenal works for an insurance company. Her project is predicting which policyholders are likely to not renew. In the morning she checks whether a promising feature is really *leakage*: information the model wouldn't have at prediction time. It is, so she removes it, and the model's accuracy drops to a more believable level. After lunch she presents results to the retention team, explaining in plain language what the model can and can't do, and proposes a test on one region before any wider rollout. She ends the day writing up the experiment design. Her day is Python, statistics, careful doubt, and explanation.

## ML engineer

Farhan works at a logistics company that uses a model to estimate delivery times. His morning alert says the model's errors have grown over the past week; he finds that a new warehouse opened and the model has never seen its routes. He schedules retraining with the new data and adds a monitoring check per warehouse. In the afternoon he packages a colleague's new model as a service, writes deployment tests, and sets up a way to roll back if it misbehaves. His day is code, monitoring, deployment, and reliability.

## Data engineer

Sneha works for a fintech company. At 8:00 she checks the overnight pipelines; one loaded zero rows from the payments system because an API credential expired, so she renews it and reruns the load before analysts arrive. She spends the morning building a new pipeline that brings support tickets into the warehouse, with checks for missing fields and duplicates. After lunch she meets the payments team to agree a *data contract*: which fields they'll send, in what format, and how they'll warn her before changing it. Her day is pipelines, failures, quality checks, and agreements with system owners.

## AI engineer

Karan works for a company that sells industrial parts. He's building an assistant that answers sales reps' questions from product manuals and price lists. In the morning he runs the assistant against a set of 150 test questions with known answers, finds that it invents a warranty period for one product line, and changes how documents are retrieved so the right manual is used. After lunch he adds a rule that any price in an answer must come from the price list, with a link. He ends the day reviewing how often reps click "not helpful". His day is Python, retrieval, evaluation, and guardrails.

## Automation analyst or integration engineer

Nisha works in the operations team of a manufacturer. Her morning goes on a report that three planners rebuild by hand every Monday: she replaces it with a Power Query refresh and a scheduled flow that emails each planner their section in the body of an email. After lunch she investigates why yesterday's purchase orders didn't reach the supplier portal; an API call failed and nobody was alerted, so she adds retries and a failure email to the team inbox. She finishes by updating the automation inventory: what runs, when, who owns it. Her day is removing manual steps and making sure automations fail loudly, not silently.

## Data architect

Vivek works for a retail group with six brands. His morning is a design review: a team wants to add a new tool for customer data, and he asks how it will match customer IDs with the existing warehouse, who will own the matching rules, and what it will cost in two years. After lunch he writes an *architecture decision record*: a one-page note of a decision, the options considered, and why one was chosen. He spends an hour with the security team on who may access sales data by region, and ends the day sketching next year's platform roadmap for the head of data. His day is questions, trade-offs, documents, and influence without direct authority.

**Real-life example: the title you'll actually see.** Many of these people's job titles don't match the role names above. Pooja's title might be "MIS executive", Nisha's "process excellence analyst", Lakshmi's "senior reporting analyst". Search job portals by the work ("Power BI", "dbt", "Power Automate") as well as by title.

---

## 8.5 Decoding a job description

---

A **job description** (JD) is written by a hiring manager, often edited by HR, and read by hundreds of applicants and, first, by software that filters CVs. Reading one well means separating what the manager needs from what got copied in from last year's posting.

Here is the full posting that Anita Rao and Meera Iyer wrote in Chapter 7's story. It's fictional, but it's typical of a real posting for a tier 1 role.

**Data Analyst (Sales Analytics and Automation)** Riverstone Supplies · head office, hybrid · 1–3 years' experience (freshers with strong projects considered)

**About the role.** You'll be Riverstone's first dedicated data analyst, working with the sales team and reporting to the Sales Head. You'll turn our sales and customer data into lists, dashboards, and reports that the team uses every week, and automate the reports we currently build by hand.

**What you'll do**

- Build and run a weekly list of at-risk customers for the sales reps.
- Build and maintain the sales dashboard in Power BI.
- Agree definitions of key numbers, such as "active customer" and net revenue, with the finance team.
- Automate recurring reports, including the Daily Sales Flash.
- Answer ad-hoc questions from sales leadership, clearly and on time.

**Must have**

- SQL: joins and aggregation; window functions a plus.
- Excel or Google Sheets: lookups and pivot tables.
- A BI tool, Power BI preferred.
- Experience automating a recurring report, with any tool.
- Clear written communication in English.

**Nice to have**

- Python (pandas).
- VBA or Google Apps Script.
- Forecasting or customer analytics.
- Experience with a CRM or ERP.

## Reading it line by line

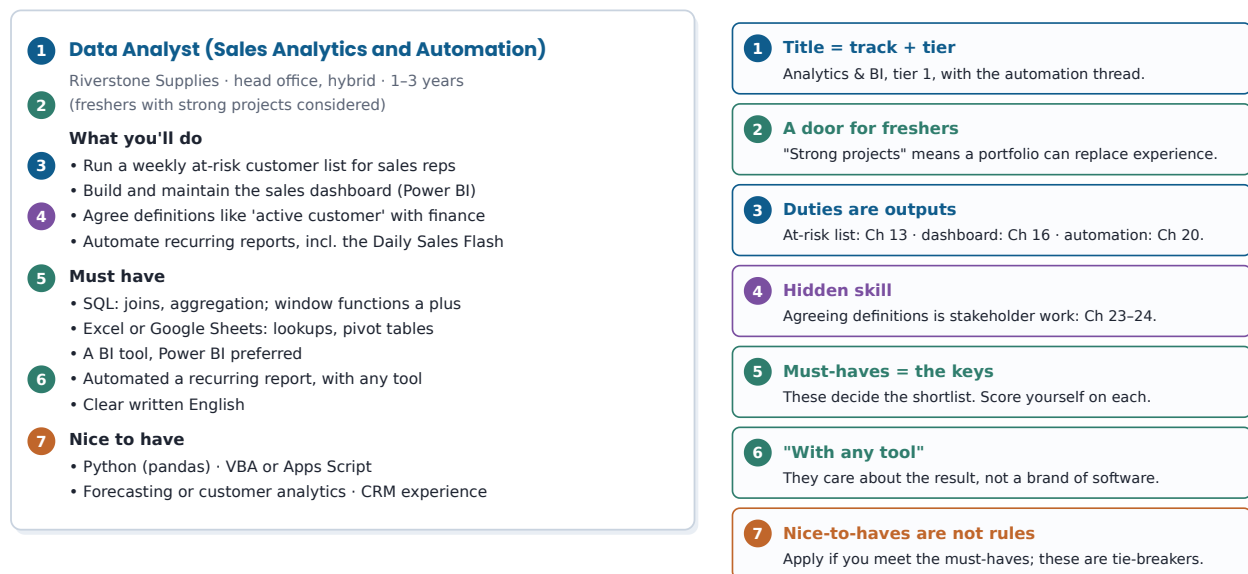


Figure 8.3 – Decoding a job description. Every line tells you something about the job, and a few lines tell you what the manager is worried about.

Work through a JD in this order.

1. **Title: find the track and the tier.** "Data Analyst" puts it on the analytics and BI track at tier 1. "Sales Analytics and Automation" tells you the domain (sales) and a thread (automation). The words in brackets often say more than the title.
2. **Experience: read the exception.** "1–3 years" is a guide; "freshers with strong projects considered" is a door. The manager is saying a portfolio can stand in for experience. Chapter 9 and Chapter 27 show how to build one.
3. **Duties: translate each into an output.** An at-risk customer list is Chapter 13's Pattern 6. A Power BI dashboard is Chapter 16. Automating the Daily Sales Flash is Chapters 19–20. When every duty maps to something you've built, you can talk about it in the interview.
4. **Find the hidden skill.** "Agree definitions with the finance team" isn't a tool. It's stakeholder work: negotiating a shared meaning across departments (Chapters 23–24). Hiring managers put lines like this in because they've felt the pain.
5. **Must-haves: these are the keys.** They decide the shortlist. Be strict with yourself here.
6. **Watch the qualifiers.** "Window functions a plus" means basic SQL is required and advanced SQL is a bonus. "Power BI preferred" means another BI tool is acceptable. "With any tool" means they care about the result: a recorded macro, an Apps Script trigger, or a Python script all count.
7. **Nice-to-haves: tie-breakers, not rules.** If you meet the must-haves, apply. Nobody expects every nice-to-have.

## Scoring your fit

A quick, honest way to judge whether to apply is to score yourself. For each skill in the JD, give yourself **0** (can't do it yet), **1** (have done it with help, or in a course), or **2** (have done it on real or realistic data without help). Must-haves count double, because they decide the shortlist.

Take Farah Khan, a sales executive at Riverstone who is interested in this role (you'll meet her properly in "In the real world").

| Skill                             | Type | Farah's score | Weight | Points    |
|-----------------------------------|------|---------------|--------|-----------|
| SQL                               | Must | 1             | 2      | 2         |
| Excel or Google Sheets            | Must | 2             | 2      | 4         |
| A BI tool                         | Must | 0             | 2      | 0         |
| Automated a recurring report      | Must | 0             | 2      | 0         |
| Written communication             | Must | 2             | 2      | 4         |
| Python                            | Nice | 0             | 1      | 0         |
| VBA or Apps Script                | Nice | 1             | 1      | 1         |
| Forecasting or customer analytics | Nice | 0             | 1      | 0         |
| CRM or ERP experience             | Nice | 2             | 1      | 2         |
| <b>Total</b>                      |      |               |        | <b>13</b> |

The maximum is 28: five must-haves at 2 points  $\times$  2 weight = 20, plus four nice-to-haves at 2 points  $\times$  1 weight = 8. Farah scores 13 of 28, or 46.4%. More useful than the total is her **must-have coverage**: her raw scores on the must-haves add up to  $1 + 2 + 0 + 0 + 2 = 5$  out of a possible 10, which is 50%.

**Reading it.** Farah isn't ready to apply today, but she's much closer than she feels. Her gaps are specific: SQL to working level (Chapters 12–13), a BI tool (Chapter 16), and one automated report she can show (Chapters 19–20). Her strengths, spreadsheets, communication, and CRM experience, are exactly the ones that are hard to teach.

**Simplification note.** This scoring is a planning tool, not a formula employers use. Its value is that it forces you to be specific about each gap, and to see which gaps matter most.

## Red flags in job descriptions

Some postings tell you more about the company than the job. Watch for these patterns.

- **A tool list longer than the duties.** Fifteen tools, from Excel to Kafka to TensorFlow, for a fresher role usually means nobody decided what the job is.
- **A senior title with junior pay or junior duties.** "Data scientist" whose duties are data entry and weekly MIS reports.
- **Every tier at once.** "Build dashboards, pipelines, ML models, and the cloud platform" for one person is four jobs.
- **No outputs at all.** If you can't tell what you'd produce in your first three months, ask in the interview.
- **Unpaid "test projects" that look like real work.** A reasonable take-home task is short and clearly an exercise; Chapter 82 shows what good ones look like.

None of these is always a reason to walk away. A small company with a messy JD might be a great place to learn. But they're questions to ask before you accept.

**Interview extra point.** Bring your decoded JD to the interview. When asked "Why this role?", name two duties from the posting, say what you've built that matches each, and ask one question about the hidden skill ("How are definitions like 'active customer' agreed today?"). Chapter 68 explains why this lands well with hiring managers.

## 8.6 What the roles pay, and how to read salary figures

Pay matters, and you deserve real numbers. You also deserve an honest warning: **salary figures online are rough, vary widely by source, and go stale within a year.** Treat everything in this section as a way to read salary data, with examples, not as a promise of what you'll earn.

### Two kinds of salary source

Salary websites build their figures in two main ways.

- **Self-reported salaries.** People enter their own pay, title, and experience. PayScale works this way. The figures reflect real pay, but only from people who chose to report it.
- **Job postings.** The site collects the salaries advertised in job ads. Indeed's salary pages work this way. The figures reflect what employers advertise, which may differ from what people accept.

Because they measure different things, the two can disagree for the same role, and neither is wrong.

### Three roles, as published in 2026

Here are figures for three roles, as shown on PayScale's India pages (updated in July 2026) when this chapter was written. **Base salary** is fixed yearly pay before bonuses; **total pay** adds bonuses and similar extras.

| Role (PayScale title) | Salary profiles | Average total pay, under 1 year | Average total pay, 1-4 years | Base salary, 10th-90th percentile | Median base salary |
|-----------------------|-----------------|---------------------------------|------------------------------|-----------------------------------|--------------------|
| Data Analyst          | 2,389           | ₹413,462                        | ₹565,999                     | about ₹289,000 to ₹1 million      | about ₹577,000     |
| Data Scientist        | 1,267           | ₹595,255                        | ₹1,005,147                   | about ₹312,000 to ₹2 million      | about ₹1 million   |
| Data Engineer         | 1,284           | ₹518,398                        | ₹810,624                     | about ₹410,000 to ₹2 million      | about ₹975,000     |

Source: PayScale India job pages for each title, pages updated 9–14 July 2026, retrieved 16 September 2026. PayScale rounds its percentile figures (for example "₹1m"), so those columns are approximate. Figures change often; check the live pages.

In Indian terms, a data analyst's average total pay in the first year works out to about ₹4.1 lakh, and a data scientist's average in years one to four to about ₹10.1 lakh.

A **percentile** tells you where a value sits in a sorted list. The 10th percentile is the value below which 10% of reported salaries fall; the 90th percentile is the value below which 90% fall. So "₹289,000 to ₹1 million" means the middle 80% of data analyst profiles reported base pay in that range. The **median** is the middle value: half earn less, half earn more.

## Reading the table carefully

**The pattern is clearer than the numbers.** Roles that need more tiers of skill (science and engineering) show higher medians and faster early growth than the analyst role in this data. The data scientist median base salary (about ₹1 million) is roughly 1.73 times the data analyst median (about ₹577,000).

**Don't read growth as a promise.** Average total pay for data analysts with 1–4 years is 36.9% higher than for those under a year:  $(₹565,999 \div ₹413,462 - 1) \times 100 = 36.9\%$ . For data scientists the difference is 68.9%, and for data engineers 56.4%. But these are *different groups of people*, not the same people a few years later. People who left the field, or never reported, aren't in the data.

**Sources disagree.** For data analysts, Indeed's India page (based on 681 salaries from job postings, updated 30 August 2026) showed an average of ₹633,625 a year. PayScale's average base salary for the same title was ₹577,472. The difference is ₹56,153, with Indeed about 9.7% higher. That's not an error: one counts advertised pay across all experience levels, the other counts reported base salaries.

**Some roles don't have reliable figures under their own title yet.** Analytics engineer, AI engineer, and integration engineer are newer titles, and many people doing that work are listed under older ones (data engineer, software engineer, BI developer). Business analyst figures mix IT business analysts, finance business analysts, and more. For these, look at live job postings for the work you'd do, not only the title, and compare several sources.

## CTC and in-hand pay

Indian offers are usually stated as **CTC** (cost to company): everything the employer spends on you in a year. CTC often includes the employer's provident fund contribution, variable or performance pay that isn't guaranteed, and sometimes insurance, gratuity, or one-time bonuses. Your **in-hand** (take-home) pay is what reaches your bank account each month after income tax, your own provident fund contribution, and other deductions. In-hand pay is always lower than  $CTC \div 12$ , and how much lower depends on how the offer is structured and on tax rules at the time. Always ask for the breakup, and ask which parts are fixed.

**Watch out: what moves pay.** City, company type, industry, and your specific skills can move pay more than the job title does. Two "data analyst" offers in the same month can differ by more than the whole range between roles in the table above. Compare offers on fixed pay, the work you'll do, and what you'll learn, not on the headline CTC alone. This is general information, not financial advice.

## How to check salaries yourself

1. Look up the role on at least two sources that use different methods (for example PayScale and Indeed), and note the date each page was updated.
2. Filter by city and experience where the site allows it, and write down the number of salaries behind each figure. A figure built from 20 salaries means much less than one built from 2,000.
3. Read five live job postings for the role in your city and note any advertised ranges.
4. Ask people in the role, respectfully, what range is typical. Most people will share a range, not their own salary.
5. Treat the result as a range, and update it every six months.

## 8.7 Reader pathways: your route through this book

Not everyone climbs to the top, and you shouldn't read every chapter with the same care. This table suggests a route for each goal. "Read fully" means work through the chapters, exercises, and projects. "Skim" means read to understand the ideas and vocabulary. "Interview chapters" lists the Part VIII chapters to prepare with.

| Goal                                     | Read fully                                          | Skim                          | Interview chapters                     |
|------------------------------------------|-----------------------------------------------------|-------------------------------|----------------------------------------|
| <b>Complete beginner, exploring</b>      | Parts 0, I                                          | Part II (first half)          | —                                      |
| <b>Data analyst</b>                      | Parts 0, I, II (all)                                | Part III (Ch 28, 30)          | 68, 69, 70, 71, 72, 73, 75, 78, 81, 82 |
| <b>Business analyst</b>                  | Parts 0, I; Ch 10–16, 19–27                         | Ch 17–18                      | 68, 69, 70, 71, 75, 76, 78, 81         |
| <b>BI developer</b>                      | Parts 0, I, II; Ch 28, 32                           | Ch 45–49, 51, 63              | 68, 69, 70, 71, 77, 78, 81             |
| <b>Analytics engineer</b>                | Parts 0–III                                         | Ch 45–49, 51                  | 68, 69, 71, 72, 77, 81                 |
| <b>Automation / integration engineer</b> | Parts 0, I; Ch 10–20, 25, 29, 34, 45–47, 51, 58, 63 | Ch 52, 55                     | 68, 69, 70, 71, 72, 76, 77, 78, 81, 82 |
| <b>Data scientist</b>                    | Parts 0–IV                                          | Part V; Ch 53–56, 58          | 68, 69, 71–75, 79, 81, 82              |
| <b>Data engineer</b>                     | Parts 0–III, V                                      | Part IV (Ch 35–39); Ch 56, 63 | 68, 69, 71, 72, 77, 78, 81, 82         |
| <b>ML / AI engineer</b>                  | Parts 0–VI                                          | Part VII                      | 68, 69, 71, 72, 74, 77, 78, 79, 81, 82 |
| <b>Data / ML architect</b>               | Everything                                          | —                             | 68, 69, 77, 78, 79, 80, 81             |
| <b>Already an analyst</b>                | Skim Parts 0–II; start fully at Part III            | —                             | Per target role                        |

Two things are worth noticing. First, **every route reads Parts 0 and I fully**, including this chapter, and every route except the beginner's goes through the analyst core. Second, the **automation and integration route** is the most spread out: it takes the spreadsheet and report automation chapters

from Part II, the software habits from Part III, the integration chapters from Part V, and the automation chapters from Parts VI and VII.

## The automation thread, tier by tier

Whichever route you take, automation grows with you. At each tier, the same business need, getting the right data to the right people and systems without manual work, is met with better tools.

| Tier                                                | What automation looks like at this level                                                                                                                                                                               | Taught in             |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <b>Foundations</b>                                  | Seeing the manual steps hidden in a business process                                                                                                                                                                   | Ch 3, 5               |
| <b>Analyst core</b>                                 | Refreshable spreadsheets; recorded macros, VBA, Office Scripts, and Google Apps Script; scheduled scripts; reports delivered by email, including in the email body; dashboard subscriptions and alerts; low-code flows | Ch 11, 16, 18, 19, 20 |
| <b>Business analyst</b>                             | Mapping processes, finding and prioritizing automation opportunities, writing automation requirements                                                                                                                  | Ch 25                 |
| <b>Advanced analytics and analytics engineering</b> | Tested, version-controlled, maintainable automations; scheduled dbt jobs; command-line scheduling                                                                                                                      | Ch 29, 32, 34         |
| <b>Engineering and integration</b>                  | Orchestrated pipelines, quality gates before delivery, pushing data back into business systems (reverse ETL, APIs, webhooks), integration platforms, RPA where there's no API                                          | Ch 45-47, 51          |
| <b>Production ML and AI</b>                         | Models and AI agents inside business workflows, with human approval where it matters                                                                                                                                   | Ch 56, 57, 58         |
| <b>Architecture</b>                                 | Designing the whole source-to-action flow as one system, choosing the right tool for each job, and governing every automation                                                                                          | Ch 60, 63             |
| <b>Interviews</b>                                   | Spreadsheet automation questions; automation, integration, and report-delivery questions and design cases                                                                                                              | Ch 70, 78             |

## 8.8 Entry routes: fresher, career switcher, internal move

Most people enter the tree at tier 1. How they get there depends on where they start.

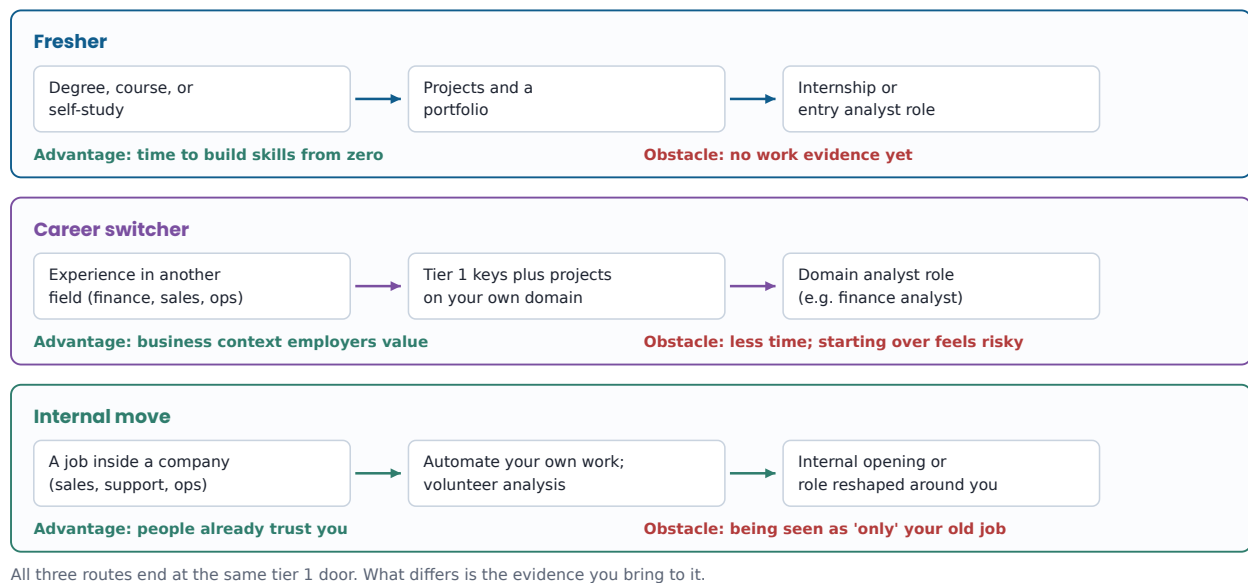


Figure 8.4 – Three entry routes. They end at the same door; what differs is the evidence you bring.

## The fresher

A **fresher** is someone applying for their first job, usually straight from a degree or course. Your challenge is evidence: the employer can't see work you've done, so you have to show it.

- **Build the tier 1 keys properly**, not a little of everything. SQL, a spreadsheet, a BI tool, and one automated report beat a certificate in ten tools.
- **Make projects that look like work**. Use realistic data (the Riverstone datasets, public government data, or a business you know), answer a real question, and write up what you'd tell a manager. The projects at the end of every chapter in Part II are designed for this, and Chapter 27 turns them into a portfolio.
- **Take internships seriously**, including short or unpaid-but-fair ones at small companies, where you often get real data and real responsibility.
- **Apply to the doors with exceptions**. Postings that say "freshers with strong projects considered" are written for you.

## The career switcher

A **career switcher** moves into data from another field: accounts, sales, operations, teaching, engineering, healthcare. Your challenge is time, and the feeling of starting over. You aren't starting over.

- **Your domain is your advantage**. An accountant who learns SQL and Power BI is a strong candidate for *finance analyst*, because they already understand what the numbers mean. Aim first for a data role in your own domain.
- **Build projects on your own domain's problems** (with made-up or public data, never your employer's confidential data). A former logistics coordinator's delivery-delay analysis tells a hiring manager more than a generic movie-ratings project.

- **Rewrite your experience as data work.** "Prepared monthly MIS reports for 12 branches" is analysis experience. Say what you produced and what decision it supported.
- **Plan in months, not weeks.** Chapter 9 is honest about how long this takes alongside a job.

## The internal move

An **internal move** is changing into a data role inside the company where you already work. For many people this is the fastest route, and it's how many data teams in mid-sized companies begin.

- **Automate your own work first.** The report you rebuild every week is your first project, and the hours you save are visible to your manager.
- **Volunteer for analysis nobody owns,** such as a question your manager keeps asking, a list that's always out of date, or a number two teams disagree on.
- **Tell your manager what you're aiming for.** Managers can't support a move they don't know about, and they often know about openings first.
- **Expect the "old job" problem.** People may keep seeing you as "the person from sales". Evidence helps: a dashboard people use, an automation that saved hours, a problem you solved with data.

**Real-life example: Meera's route.** Meera Iyer (Chapter 1) joined Riverstone as a sales coordinator, not as an analyst. By Chapter 7 she was the person everyone asked for numbers, because she asked good questions about the data and fixed the reports she inherited. That's an internal move in progress: the job title hasn't changed yet, but the work already has.

## 8.9 Using the tree: pick your door, then learn backward

Put the chapter together and planning your next step takes four moves.

1. **Pick one door.** Choose the role you want next, not the one you want in ten years. Use Chapter 7 and section 8.4 to check that you'd enjoy the day, not only the title.
2. **Find its keys.** Take the role's column in the skills matrix (section 8.3), then check it against five real job descriptions for that role in your city (section 8.5).
3. **Score yourself and list the gaps.** Use the scoring in section 8.5. Order the gaps: must-haves first, then the useful skills that appear most often in the postings.
4. **Map gaps to chapters and projects.** Use the pathways table (section 8.7) to find where each gap is taught, and plan one project that shows each must-have skill.

Then apply when your must-have coverage is strong, and keep climbing on the job. When you're settled, pick the next door.

## Common mistakes and how to spot them

| Mistake                                        | Symptom                                                                        | Fix                                                                             |
|------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Learning tools because they're trending        | Many half-finished courses; no door you're ready for                           | Pick one door and learn its keys first (section 8.9)                            |
| Trying to learn the whole tree before applying | Two years of study, no applications                                            | A role sits at the first tier that makes you credible; apply when you get there |
| Treating tiers 3 and 4 as a ladder             | Feeling you must do data science before engineering (or the reverse)           | They're peers; choose by the work you enjoy                                     |
| Skipping the analyst core                      | Models or pipelines built on questions you can't frame or data you can't check | Build tier 1 properly; every branch rests on it                                 |
| Reading "nice to have" as "must have"          | Never applying, because you don't meet every line                              | Score must-haves strictly; treat nice-to-haves as tie-breakers                  |
| Reading "must have" as "nice to have"          | Many applications, few shortlists                                              | Close must-have gaps before applying widely                                     |
| Searching only by job title                    | Missing roles called "MIS executive" or "process excellence analyst"           | Search by the work and tools too (section 8.4)                                  |
| Comparing offers by headline CTC               | Accepting the "bigger" offer and getting less in hand                          | Ask for the breakup; compare fixed pay and the work                             |
| Treating an online salary figure as a promise  | Disappointment, or turning down fair offers                                    | Use ranges from two sources, note the date and sample size (section 8.6)        |
| Building generic projects                      | Portfolio looks like everyone else's                                           | Use realistic business questions, ideally in your own domain                    |
| Switchers hiding their old career              | CV looks like a fresher's                                                      | Rewrite past work as data work; your domain is an advantage                     |
| Waiting for an internal role to be announced   | Openings filled before you hear of them                                        | Automate your work, volunteer analysis, and tell your manager your goal         |

## In the real world: Farah's internal move

Farah Khan had been a sales executive at Riverstone for four years. She knew the hospitality customers better than anyone: which caterers ordered before wedding season, which hotel buyer needed a call before Diwali. When Anita pinned the new *Data Analyst (Sales Analytics and Automation)* posting on the notice board, Farah read it three times and then walked past it.

Meera noticed. At lunch she asked, "Are you going to apply?"

"I'm a sales executive," Farah said. "I use Excel. That's not the same thing."

"Let's check," Meera said, and pulled the posting up on her laptop. They went through it the way section 8.5 describes. Farah scored herself: 2 for spreadsheets ("I run my whole customer list in Sheets, with lookups and a pivot"), 2 for written communication, 2 for CRM experience. Then 1 for

SQL, because she'd done an online course but never used it on real data; 1 for Apps Script, because she'd once recorded a macro to format her weekly report; and 0 for a BI tool, automating a report, Python, and forecasting.

Farah looked at the total, 13 of 28, and shrugged. "Less than half."

"Look at the must-haves instead," Meera said. "You're at half of those already. And look at *which* half you have. Anita can teach someone Power BI in a few months. She can't teach someone which caterers go quiet in the monsoon, and why."

They turned the gaps into a plan. Meera was careful not to promise a timeline; Farah had a full-time job, and the honest answer was months, not weeks.

- **SQL to working level:** Chapters 12 and 13 of this book, practiced on Riverstone's own practice databases, and then on questions from her own customers.
- **A BI tool:** Chapter 16, building a dashboard of her own region's customers.
- **One automated report she could show:** her weekly customer follow-up list, rebuilt as a Google Sheet that refreshes itself and emails her the list every Monday (Chapter 19).

Farah also asked the question she'd been avoiding. "What happens to my sales targets while I'm doing this?"

"Talk to Anita," Meera said. "Tell her you're aiming for the role. She'd rather grow someone who knows the customers than hire a stranger."

The conversation with Anita was shorter than Farah expected. Anita didn't promise her the job. The posting would stay open, and she'd interview other candidates on the same must-haves. But she agreed that Farah could spend Friday afternoons helping Meera with the at-risk customer list, and she asked Farah to bring the Monday email to the next sales review once it worked.

Six weeks later, the email worked. It wasn't sophisticated: a query, a sheet, and a time-driven trigger. But every Monday at 9 a.m. it listed Farah's customers who hadn't ordered in their usual rhythm, and in the first month it caught two she would have missed. At the sales review, Vikram asked if his team could have the same email.

### **What made this work.**

- **Farah scored herself against the real posting,** not against a vague idea of "being technical". Her gaps became three specific items.
  - **She used her domain as an advantage,** and built her first project on her own customers.
  - **She made her goal visible,** and turned an internal move from a hope into an agreement about what evidence would count.
  - **Nobody over-promised.** The role wasn't guaranteed, and the timeline was honest. The evidence she built would count here or anywhere else.
-

## Tools

---

- **A spreadsheet** (Excel or Google Sheets) for your skills matrix and gap score, or paper if you prefer.
  - **Two or more job portals.** Save postings as PDFs or screenshots; they disappear.
  - **Two salary sources that use different methods**, such as PayScale (self-reported) and Indeed (job postings), plus your city filter. Note the date on every figure you record.
  - **Companion files:** none for this chapter. The skills matrix values are listed in `checks/ch08_check.py` and in `figures/make_figs08.py`, if you'd like to rebuild or change it.
- 

## The project: your door plan

---

**Goal:** a two-page plan for your next role, grounded in real job postings. It's the most useful document you'll write in Part I, and a good one makes interviews easier, because you'll already know what the employer wants and what you can show.

**Option A: your own target.** Use the role you want next, in the city or remote market you'll apply to.

**Privacy reminder.** If you're planning an internal move, don't include confidential details about your employer, colleagues, or internal postings in anything you share publicly.

**Option B: Farah's plan.** Use the Riverstone posting in section 8.5 and Farah's scores, and write the plan she'd bring to Anita.

### Steps

1. **Pick one door.** Write the role and one sentence on why, using the day in the life in section 8.4.
2. **Collect five job descriptions** for that role. For Option B, use the Riverstone posting plus four real postings for similar roles.
3. **Decode each one** with the seven steps in section 8.5. Highlight must-haves in one color and nice-to-haves in another.
4. **Build a skills table** in a spreadsheet: skills down the side, postings across the top. Mark each skill as must, nice, or absent in each posting. Add a column counting how many postings list each skill as a must-have.
5. **Score yourself** 0, 1, or 2 on every skill, and calculate your score and must-have coverage for the posting you like most.
6. **List your gaps in order:** must-haves that appear in most postings first.
7. **Map each gap to chapters and one project** that would prove the skill, using the pathways table in section 8.7.
8. **Check pay** for the role using the steps at the end of section 8.6, and record the range, sources, dates, and sample sizes.
9. **Choose your entry route** (section 8.8) and write the three things you'll do differently because of it.

### Stretch goals

- Repeat steps 4–6 for a second, higher door, and mark which gaps the two doors share.
  - Ask one person in your target role to review your plan, and note what they'd change.
  - Set a date, three months out, to re-score yourself against the same postings.
- 

## You've got it when...

---

- ☐ You can explain "skills are keys, roles are doors" and why it changes what you learn next.
  - ☐ You can name the seven tiers in order, say which part of the book teaches each, and explain why tiers 3 and 4 are peers.
  - ☐ You can place each of the ten roles at the first tier that makes someone credible for it.
  - ☐ You can read your target role's column in the skills matrix and list its core skills.
  - ☐ You can describe an ordinary day in at least three roles, and say which you'd enjoy.
  - ☐ You can decode a job description into title, duties as outputs, hidden skills, must-haves, and nice-to-haves.
  - ☐ You can score your fit and calculate your must-have coverage.
  - ☐ You can read a salary table with percentiles, and explain why two sources disagree.
  - ☐ You can explain CTC versus in-hand pay.
  - ☐ You can choose a reading pathway through this book for your goal.
  - ☐ You've written down your next door and its three most important gaps.
- 

## Recap

---

- **Skills are keys; roles are doors.** Learn skills for the roles they unlock, not because they're popular.
- The **career tree** has seven **tiers** that match the book's parts: foundations (0), the analyst core (1), advanced analytics and analytics engineering (2), data science (3), engineering and integration (4), production ML and AI (5), and architecture and leadership (6). **Tiers 3 and 4 are peers.**
- A role sits at the **first tier that makes you a credible applicant**, and you keep every skill below it.
- The **skills matrix** shows each role's **core** and **useful** skills. SQL helps open every door; machine learning is core for only two.
- A **day in the life** matters as much as the matrix: choose work you'd enjoy on an ordinary day.
- **Decoding a job description** means finding the track and tier, turning duties into outputs, spotting hidden skills, and separating **must-haves** from **nice-to-haves**. **Must-have coverage** is the most useful fit score.
- **Salary figures** differ by method (self-reported versus job postings), date, sample size, city, and company. Read **percentiles** and **medians**, compare two sources, and know **CTC** from **in-hand** pay.
- **Reader pathways** show which parts of this book to read fully or skim for each goal, and automation grows with every tier.

- There are three **entry routes**: **fresher**, **career switcher**, and **internal move**. They end at the same door with different evidence.
- 

## Practice exercises

---

### Warm-up

1. List the seven tiers in order, with the part of the book that teaches each. Which two tiers are peers?
2. Place each role at the first tier that makes someone credible for it: (a) automation analyst, (b) analytics engineer, (c) ML engineer, (d) data architect, (e) business analyst, (f) integration engineer.
3. Using the skills matrix in section 8.3: (a) For which roles is machine learning a core skill? (b) For which roles is Python a core skill? (c) Which skill is core or useful for every role?

### Core

4. Classify each line from a job posting as a must-have, a nice-to-have, a hidden skill, or a signal about who can apply: (a) "Experience with Tableau or Power BI." (b) "Exposure to Python is an advantage." (c) "You'll work with regional managers to agree monthly targets." (d) "Final-year students with internship experience may apply." (e) "Strong SQL is essential."
5. Rahul scores himself against the Riverstone posting in section 8.5. Must-haves: SQL 0, spreadsheets 2, BI tool 1, automated a report 1, written communication 1. Nice-to-haves: Python 1, VBA or Apps Script 0, forecasting 2, CRM or ERP 2. Calculate his total score out of 28, the percentage, and his must-have coverage. Which single gap would you tell him to close first, and why?
6. Using the PayScale table in section 8.6, calculate how much higher average total pay is for data engineers with 1–4 years than for those under a year, as a percentage. Then give one reason why this isn't the pay rise a new data engineer should expect after a few years.
7. A friend says: "*Indeed says data analysts earn ₹633,625 on average, but PayScale says ₹577,472. One of them must be wrong.*" Explain why both can be right, giving two differences between the sources.
8. Explain the difference between CTC and in-hand pay in three sentences, as you would to a friend comparing two offers.
9. Using the reader pathways table, list what someone aiming to be a BI developer should read fully, what they should skim, and which interview chapters they should prepare with.

### Stretch

10. Here's a posting: "*Data Scientist (Fresher). Must know Excel, SQL, Power BI, Tableau, Python, R, Spark, Hadoop, Kafka, AWS, Azure, GCP, TensorFlow, PyTorch, and LLMs. Duties: daily data entry of invoices and preparing weekly MIS reports for the accounts team.*" List at least three red flags, say what the job really is (role and tier), and write two questions you'd ask before applying.

- l1. Neha, a sales executive with strong Excel skills and three years of customer contact, wants to become a data analyst. Recommend an entry route and write a five-step plan using section 8.8 and section 8.9. Include which chapters of this book close which gaps.
- l2. Using the skills matrix, count the skills that data analyst and BI developer both need at least some of (core or useful in both columns), and do the same for data analyst and data engineer. What do the two counts suggest about which move is shorter, and which skills make the difference?

### Think about it (no calculation needed)

- l3. The tree says a role sits at the first tier that makes you credible. Why is that more useful for planning than placing each role at the highest tier its best practitioners reach?
- l4. Could someone skip tier 1 and go straight into data engineering, for example from a software engineering job? What would they gain, and what would they risk missing?

---

## Key terms

key · door · credible applicant · career tree · tier · MIS (management information system) · BI analyst · senior analyst · skills matrix · core skill · useful skill · leakage · data contract · architecture decision record · job description (JD) · must-have · nice-to-have · hidden skill · must-have coverage · self-reported salary · base salary · total pay · percentile · median · CTC (cost to company) · in-hand pay · reader pathway · fresher · career switcher · internal move

*(All terms are defined in the Glossary, Appendix A.)*

---

## Where this leads

- **Chapter 9, How Expertise Actually Forms**, is honest about how long each tier takes, and shows how to practice deliberately, build a portfolio as you learn, find feedback and mentors, and get through plateaus.
- **Part II, The Analyst (Chapters 10–27)**, is tier 1: the keys for data analyst, business analyst, BI analyst, and automation analyst. **Chapter 27** turns your projects into a portfolio.
- **Chapters 23–24** teach the stakeholder skills behind "hidden skill" lines in job descriptions; **Chapter 25** covers the business analyst track in depth.
- **Chapter 66** returns to careers from the other side of the desk: building and hiring for data teams.
- **Part VIII, Chapter 68, How Data Hiring Works**, explains the interview rounds for each role, CVs, portfolios, and referrals. **Chapter 81** covers behavioral questions and offer conversations, including negotiating on CTC. **Chapter 82** shows what fair take-home assignments look like.

## Answers to practice exercises

---

(In the finished book these move to Appendix G.)

1. Tier 0, foundations (Parts 0 and I); tier 1, the analyst core (Part II); tier 2, advanced analytics and analytics engineering (Part III); tier 3, data science and machine learning (Part IV); tier 4, engineering and integration (Part V); tier 5, production ML and AI (Part VI); tier 6, architecture and leadership (Part VII). **Tiers 3 and 4 are peers.** The common wrong answer puts data engineering at tier 3, as the first edition did; the tier numbers now follow the book's parts.
2. (a) Automation analyst: tier 1. (b) Analytics engineer: tier 2. (c) ML engineer: tier 5. (d) Data architect: tier 6. (e) Business analyst: tier 1. (f) Integration engineer: tier 4. The trap is (a) and (f): both are on the automation track, but at different tiers, because an integration engineer needs the pipeline and API skills of Part V.
3. (a) Data scientist and ML engineer. (b) Data scientist, ML engineer, data engineer, AI engineer, and the automation and integration roles. (c) SQL, which is core for six roles and useful for the other four.
4. (a) Must-have, with a qualifier: either BI tool is acceptable. (b) Nice-to-have: "an advantage". (c) Hidden skill: negotiating targets with managers is stakeholder work, not a tool. (d) Signal about who can apply: a door for students with internship evidence. (e) Must-have: "essential".
5. Must-have points:  $(0 + 2 + 1 + 1 + 1) \times 2 = 5 \times 2 = 10$ . Nice-to-have points:  $1 + 0 + 2 + 2 = 5$ . Total **15 of 28 = 53.6%**. Must-have coverage: 5 of 10 = **50%**, the same as Farah's, even though his total is higher, because his extra points come from nice-to-haves. Close **SQL** first: it's a must-have, he scores 0, and it's the one skill every role on the tree uses. His forecasting score won't get him shortlisted without it. The common wrong answer is to celebrate the higher total; must-haves decide the shortlist.
6.  $(\text{₹}810,624 \div \text{₹}518,398 - 1) \times 100 = \textbf{56.4\%}$  higher. It isn't a promised rise because the two figures come from **different groups of people**, not the same people over time. The 1–4 year group also only includes people still in the role who chose to report, and pay depends heavily on city, company, and skills. Any one of these is a good reason.
7. Both can be right because they measure different things. (1) **Method:** Indeed's figure comes from salaries advertised in job postings; PayScale's comes from salaries people report themselves. (2) **What's counted:** the PayScale figure quoted is average base salary, while advertised figures may include other pay. The sources also differ in **sample** (681 postings versus 2,389 profiles), **date** (updated 30 August 2026 versus July 2026), and the mix of experience and cities. The gap is ₹56,153, about 9.7%, which is small next to the range within either source.
8. A sample answer: "CTC is everything the company says it spends on you in a year, including things like the employer's provident fund share and bonuses that aren't guaranteed. In-hand pay is what actually reaches your bank account each month, after tax and your own deductions, so it's always less than CTC divided by 12. When you compare offers, ask for the breakup and compare the fixed, monthly in-hand amount, not only the CTC headline."
9. **Read fully:** Parts 0, I, and II, plus Chapters 28 and 32. **Skim:** Chapters 45–49, 51, and 63. **Interview chapters:** 68, 69, 70, 71, 77, 78, and 81.

**10. Red flags:** (1) a **tool list far longer than the duties**, covering every tier from spreadsheets to cloud to LLMs; (2) a **senior title with junior duties**: invoice data entry and MIS reports aren't data science; (3) a **fresher role expected to know tools from tiers 1–5**, which nobody entering the field has. The real job is closest to a **reporting or MIS assistant (tier 0 to tier 1)**, with some data entry. Useful questions: "What would I produce in my first three months?" and "Which of the listed tools are used by the team today?" Other good questions ask about who the role reports to, or whether there's a path into analysis. The job might still be a reasonable first step if the pay and learning are fair, but not under the belief that it's a data science role.

**11. Recommended route:** **internal move** if Neha's company has analysis work she can take on, otherwise **career switcher** into a sales analyst role, using her customer knowledge as her advantage. A five-step plan: (1) **Pick the door:** data analyst, ideally sales analytics. (2) **Find the keys:** the data analyst column (business sense, communication, spreadsheets, SQL, BI) checked against five postings. (3) **Score and list gaps:** likely SQL (Chapters 12–13), a BI tool (Chapter 16), and one automated report (Chapters 19–20); her spreadsheets and business sense are strengths. (4) **Build evidence on her own domain:** automate her weekly customer report and build a dashboard of her region, using non-confidential or practice data if she'll show it publicly. (5) **Make it visible:** tell her manager her goal, volunteer for an analysis nobody owns, and re-score herself in three months. Any plan that orders must-have gaps first and ties each to a project is acceptable.

**12. Data analyst and BI developer: 7 shared skills** (business and domain sense, communication and storytelling, spreadsheets, SQL, BI and dashboards, spreadsheet and report automation, requirements and process mapping). **Data analyst and data engineer: 4** (business sense, communication, SQL, and Python). Analyst to BI developer is the shorter move: the analyst already has most of what's needed, and the main new core skill is data modeling. Analyst to data engineer needs several skills the analyst column doesn't have at all: data modeling, pipelines and orchestration, cloud, APIs and integration, and Git.

**13.** Because it tells you when you can apply. If each role were placed where its best people end up, a data analyst would sit at tier 6 (some senior analysts have architect-level judgment), and every beginner would conclude they need years of study before applying for anything. Placing roles at the first credible tier gives each door a clear, reachable set of keys, and the rest can be learned on the job, which is how most people climb.

**14.** Yes, it's possible, and common. A software engineer already has much of what data engineering needs: Python, Git, cloud, APIs, and software habits. They'd gain speed. They'd risk missing the tier 1 skills that make data engineering *useful*: business sense, understanding what the numbers mean, knowing what analysts need, and the habit of checking a total against a source. A data engineer without them can build fast, reliable pipelines that deliver the wrong numbers on time. The fix is to learn the analyst core alongside the move, not to skip it.